

Module : 2

- Linear Search Time Complexity:
 - Best Case: $O(1)$
 - Average Case: $O(n)$
 - Worst Case: $O(n)$
- Binary Search Time Complexity:
 - Best Case: $O(1)$
 - Average Case: $O(\log n)$
 - Worst Case: $O(\log n)$

Exercise 2: E-commerce Platform Search Function Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. Understand Asymptotic Notation:

- Explain Big O notation and how it helps in analyzing algorithms.
- Describe the best, average, and worst-case scenarios for search operations.

2. Setup:

- Create a class **Product** with attributes for searching, such as **productId**, **productName**, and **category**.

3. Implementation:

- Implement linear search and binary search algorithms.
- Store products in an array for linear search and a sorted array for binary search.

4. Analysis:

- Compare the time complexity of linear and binary search algorithms.

- Discuss which algorithm is more suitable for your platform and why.

CODE :

```

Main.cs
1  using System;
2  class Product
3  {
4      public int ProductId;
5      public string ProductName;
6      public string Category;
7
8      public Product(int id, string name, string category)
9      {
10         ProductId = id;
11         ProductName = name;
12         Category = category;
13     }
14 }
15 class Program
16 {
17     static Product LinearSearch(Product[] products, int id)
18     {
19         foreach (Product p in products)
20         {
21             if (p.ProductId == id)
22                 return p;
23         }
24         return null;
25     }
26 }

```

```

27 // Binary Search
28 static Product BinarySearch(Product[] products, int id)
29 {
30     int left = 0, right = products.Length - 1;
31     while (left <= right)
32     {
33         int mid = (left + right) / 2;
34         if (products[mid].ProductId == id)
35             return products[mid];
36
37         if (products[mid].ProductId < id)
38             left = mid + 1;
39         else
40             right = mid - 1;
41     }
42
43     return null;
44 }
45
46 static void Main()
47 {
48     Product[] products =
49     {
50         new Product(101, "Laptop", "Electronics"),
51         new Product(102, "Phone", "Electronics"),

```

```

static void Main()
{
    Product[] products =
    {
        new Product(101,"Laptop","Electronics"),
        new Product(102,"Phone","Electronics"),
        new Product(103,"Shoes","Fashion"),
        new Product(104,"Watch","Accessories"),
        new Product(105,"Book","Education")
    };

    int searchId = 103;

    Product result1 = LinearSearch(products, searchId);

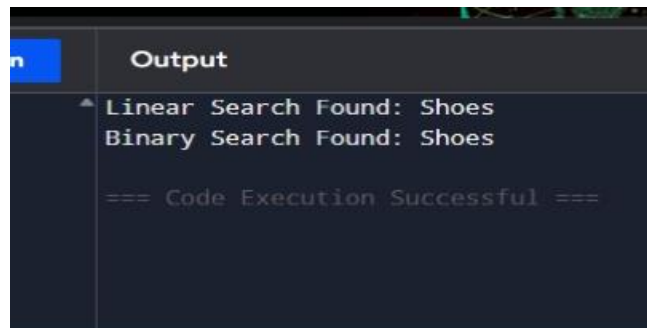
    if (result1 != null)
        Console.WriteLine("Linear Search Found: " + result1.ProductName);

    Product result2 = BinarySearch(products, searchId);

    if (result2 != null)
        Console.WriteLine("Binary Search Found: " + result2.ProductName);
}
}

```

OUTPUT :



```

Output
Linear Search Found: Shoes
Binary Search Found: Shoes

=== Code Execution Successful ===

```

Exercise 7: Financial Forecasting Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Steps:

1. Understand Recursive Algorithms:

- Explain the concept of recursion and how it can simplify certain problems.

2. Setup:

- Create a method to calculate the future value using a recursive approach.

3. Implementation:

- Implement a recursive algorithm to predict future values based on past growth rates.

4. Analysis:

- Discuss the time complexity of your recursive algorithm.

Explain how to optimize the recursive solution to avoid excessive computation.

Analysis

- Time Complexity: $O(n)$
- Space Complexity: $O(n)$ (due to recursive call stack)

CODE :

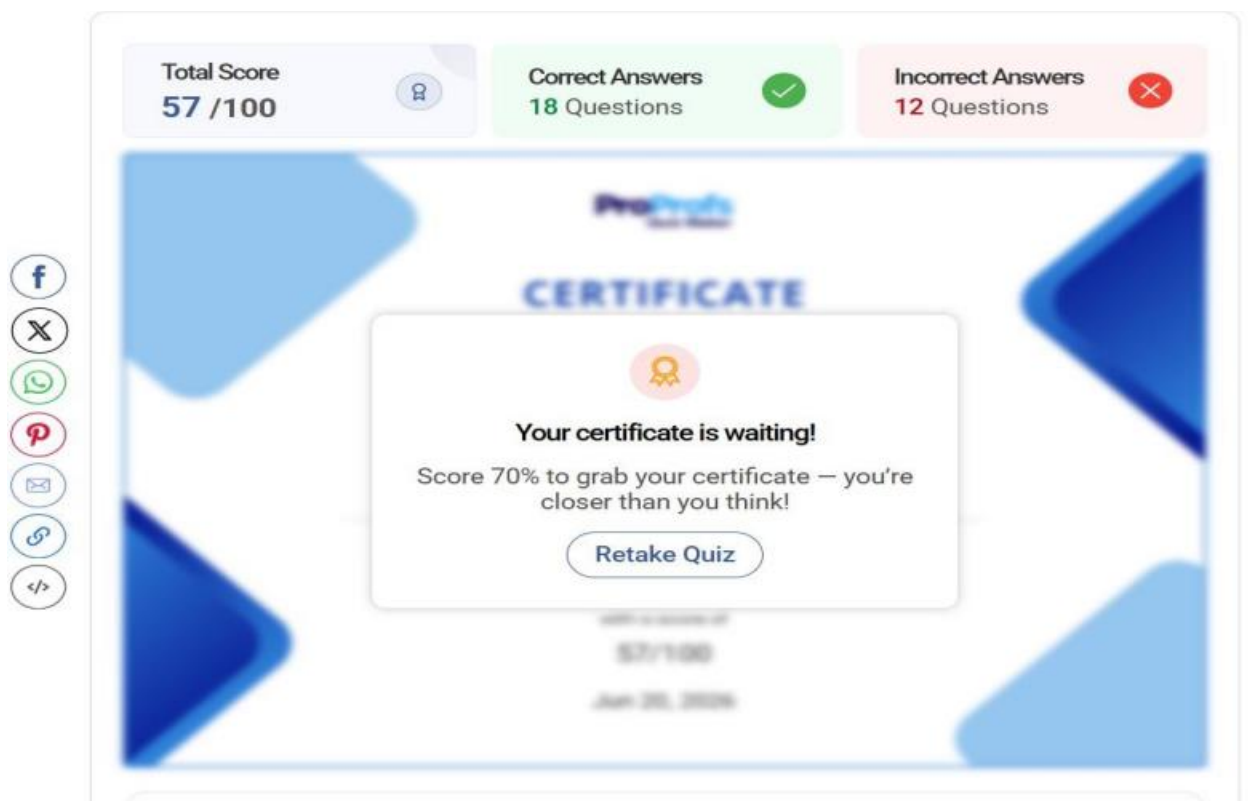
```
Online Compiler
1 using System;
2 class Program
3 {
4     static double Forecast(double currentValue, double growthRate, int years)
5     {
6         if (years == 0)
7             return currentValue;
8         return Forecast(currentValue, growthRate, years - 1)
9             * (1 + growthRate);
10    }
11    static void Main(string[] args)
12    {
13        double presentValue = 10000;
14        double growthRate = 0.10; // 10%
15        int years = 5;
16
17        double futureValue = Forecast(presentValue, growthRate, years);
18
19        Console.WriteLine("Present Value : " + presentValue);
20        Console.WriteLine("Growth Rate : " + (growthRate * 100) + "%");
21        Console.WriteLine("Years : " + years);
22        Console.WriteLine("Future Value : " + futureValue);
23    }
24 }
```

OUTPUT:

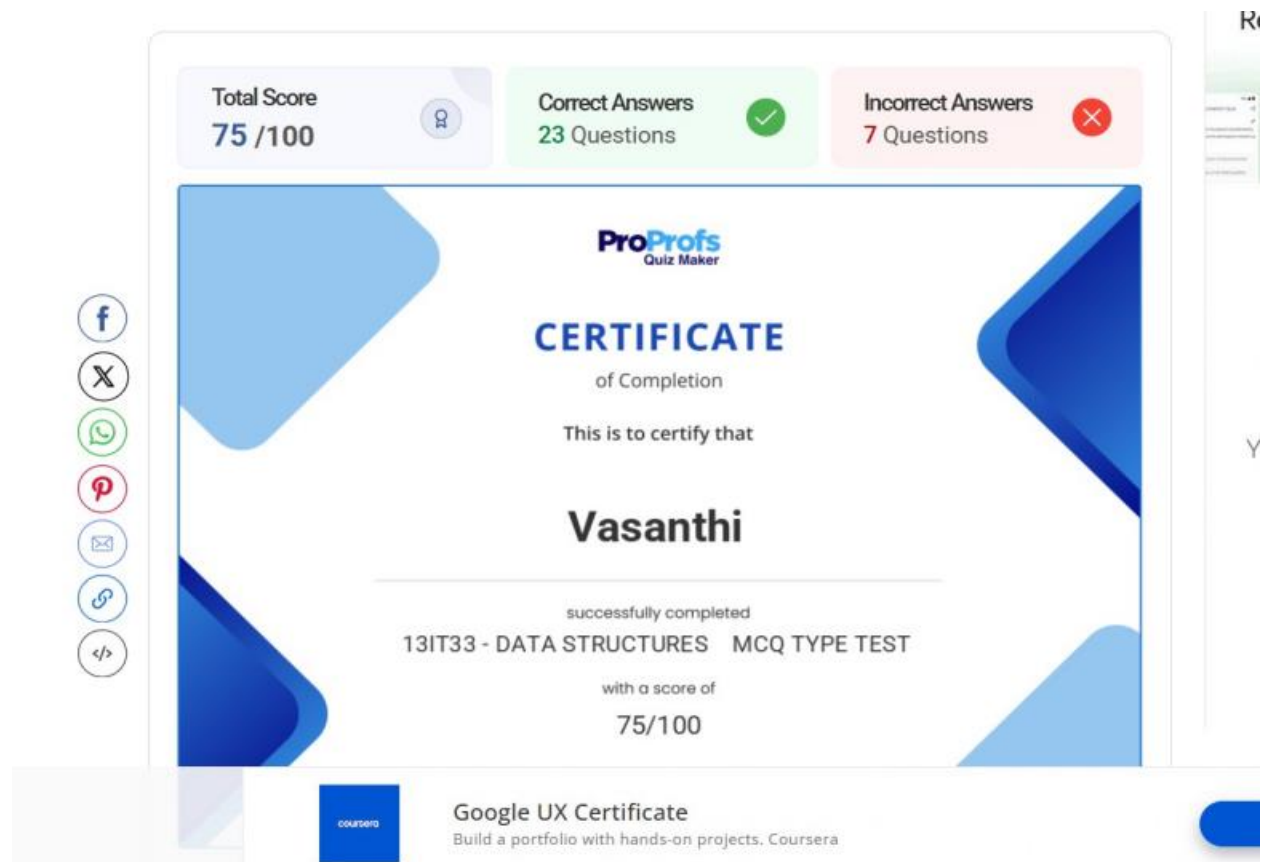
```
Output
Present Value : 10000
Growth Rate   : 10%
Years         : 5
Future Value  : 16105.1

=== Code Execution Successful ===
```

Quiz:



Before



AFTER PRACTICE: